CASE STUDY — SUPPORTVECTORS AI AGENTS BOOTCAMP CAPSTONE

The Disclosure Verifier

An agentic system that checks whether a company's own words match its own numbers — extracting quantitative claims from SEC filing prose and reconciling each one against the structured data the company itself reported, **with a citation back to the exact filing.**

Status Phases 0–7 complete **Tests** 216 passing, CI green

Repo github.com/asim-aa/disclosure-verifier

"Revenue for fiscal year 2026 was $215.9 billion, up 65% from a year ago."

extracted → `{metric: Revenue, value: 215,900,000,000, type: absolute}`

extracted → `{metric: Revenue, value: 65.0, type: growth_pct}`

reconciled → `CONSISTENT` EDGAR reports $215,938,000,000 — 0.02% off

reconciled → `INCONSISTENT` EDGAR implies 46.09% growth, not 65%

01 DSPy optimization, measured against a baseline

The capstone's Pillar 2 requirement: at least one prompt signature optimized with DSPy, scored against a hand-written baseline — not just used, *proven*. Ground truth is 78 examples / 161 claims / 9 true negatives, hand-labeled from real MSFT and NVDA filings before any DSPy code was written, to keep the measurement honest.

EXTRACTOR	PRECISION	RECALL	F1
Hand-written baseline prompt	0.711	0.750	0.730
DSPy zero-shot (ChainOfThought, no demos)	0.763	0.806	0.784
DSPy optimized (BootstrapFewShot)	**0.763**	**0.806**	**0.784**

Held-out test set, 16 examples never seen during optimization. Delta vs. baseline:
+0.054 F1

The honest framing matters more than the win — including a result that didn't survive its own honesty check. Switching the signature from `Predict` to `ChainOfThought` — reasoning before committing to structured output — took zero-shot DSPy from underperforming the baseline (an earlier run measured 0.676 F1 on the bare `Predict` signature) to clearly beating it (0.784), before any optimization ran. `BootstrapFewShot` optimization on top of that produced *exactly identical* pooled numbers: same precision, same recall, the same 29/9/7 true-positive/false-positive/false-negative split. Verified this wasn't an evaluation bug — the compiled program is a genuinely separate instance, and the optimizer log confirms 4 real demonstrations were bootstrapped.

> **CHECKED AGAINST ITS OWN NOISE FLOOR**
>
> A 16-example held-out set produces 45 claim-level tp/fp/fn decisions — small enough that `eval/run_comparison.py` now computes the sampling noise floor (`SE(p) = sqrt(p(1−p)/n)` , ~95% half-width `1.96 · SE`) alongside every delta, rather than letting a modest F1 gain read as a settled result. At n=45, that half-width is ±0.120 — wider than the +0.054 delta above. The pooled win is not distinguishable from sampling noise at this test set size; a bigger held-out set is what it would take to resolve this, not a cleaner summary sentence.

A stratified, per-`comparison_type` breakdown adds a second reason the pooled number understates the picture: optimization moved the two claim categories in

opposite directions. `absolute` claims improved (0.625 → 0.788 F1), while `absolute_change` claims got worse (0.429 → 0.308) — a regression a pooled average hides because `growth_pct` claims (1.000 F1 throughout) dominate the sample and keep the aggregate looking flat-to-positive. Reasoning mattered more than optimization for this task, and the optimization step itself is a wash at best and a net negative for one category at worst — a more interesting and more honest finding than a clean "optimization wins" story would have been.

TRIED THE NEXT OPTIMIZER UP, HONESTLY

`BootstrapFewShot` adding nothing was the reason to try `dspy.GEPA` — a reflective optimizer that rewrites the instruction itself, guided by *why* specific attempts failed rather than a bare pass/fail score. Caveat stated before the result, not after: GEPA's reflection step is meant to be guided by a model stronger than the one being optimized, and this project has exactly one LLM endpoint (gpt-oss-20b) — it reflects on its own failures here, not a stronger model's. Real run, `auto="light"` , 444 metric calls, 37 minutes on the local single-GPU server: F1 `0.784 → 0.800`, a `+0.016` delta *inside* the ±0.117 noise floor at this sample size — not distinguishable from chance, same honest conclusion as `BootstrapFewShot` . The per-category read is more telling than the pooled number: `growth_pct` reached a perfect 1.000 F1 and `absolute` improved to 0.857, but `absolute_change` collapsed to 0.167 — a trade-off, not a clean win.

THE REPEATED-TOUCH CAVEAT, FINALLY CLOSED

Every number above has one honest asterisk: the same 16-example test slice was scored on every run — baseline, zero-shot, `BootstrapFewShot` , GEPA — so every delta was "the best estimate from a set we've looked at many times," never a clean holdout result. Closed directly: 25 fresh examples / 59 claims, hand-labeled by reading real AMZN and AAPL 10-K MD&A text — two companies absent from the training data entirely, scored for the first time by this run. The result actually *moves*: baseline collapsed on genuinely new companies (`0.730 → 0.344` F1, 44 false positives in one category), while zero-shot held up (`0.784 → 0.752`). DSPy vs. baseline is now a real, **noise-floor-clearing** result — `+0.355` F1 at n=82, noise floor ±0.099 — resolved for the first time instead of perpetually "inconclusive at this sample size." A new question opened with it: `BootstrapFewShot` didn't just fail to help on the fresh set, it measurably *hurt* (`0.752 → 0.698`) — consistent with its 4 demos overfitting to MSFT/NVDA's phrasing, though at n=25 that specific drop is itself still inside its own noise floor. One finding closed, a sharper one opened — not a tidier story, a truer one.

02 RLVR/GRPO fine-tuning, checked the same way

The capstone's Pillar 4 requirement: reinforcement learning from verifiable rewards — training a small open model to reason through the arithmetic the Numerical Reconciler already does deterministically, using the Reconciler itself as the reward, audited first (15/15 correct, zero false-"consistent" results — see finding 08 below). `Qwen2.5-7B-Instruct`, QLoRA rank 32 via Unsloth, a single RTX 4090. Same discipline as Pillar 2: baseline vs. trained, checked against the noise floor, not just reported as a bare delta.

	ACCURACY	MEAN REWARD	FALSE-CONSISTENT
Baseline (zero-shot)	0.766	0.633	0.148
Trained (1 epoch, 1,300 GRPO steps)	**0.855**	**0.826**	**0.036**

Held-out test set, 304 examples. Delta vs. baseline: **+0.089 accuracy**

Unlike Pillar 2's optimization delta, this one survives its own noise-floor check: at n=304 the combined 95% half-width is ±0.062 — narrower than the +0.089 delta. The false-consistent rate — the dangerous failure mode, a model confidently endorsing a wrong claim — dropped from 0.148 to 0.036, also clearing its own noise floor (±0.045 against a –0.112 delta). Format failures (no parseable verdict) went from 1/304 to 0/304.

COMPARISON_TYPE	BASELINE	TRAINED
absolute	0.949	0.975
absolute_change	**0.589**	**0.821**
bps_change	0.233	0.400
growth_pct	0.810	0.870

`absolute_change` shows the largest per-category gain, also noise-floor-clearing at this stratum's n=56. `bps_change` — two ratios, then a basis-point difference, the hardest task in the set — moved from 7/30 to 12/30 correct, but at n=30 that's not provable on its own; a first, partial run (150 steps, ~31 minutes) had left it at an exact, unchanged null. Directionally real, not yet statistically defensible — stated honestly rather than rounded up.

Same held-out example, both models, greedy decoding — reasoning *quality*, not a verdict flip (both reach the correct answer here):

```
BASE MODEL

Step 1: Calculate the actual change in revenue.
Actual change = Current period value - Prior period value
              = 274,515,000,000 USD - 2260,174,000,000 USD
              = 14,341,000,000 USD
Step 2: Check if the claimed value falls within the tolerance range.
Tolerance = 0.01 * max(2260,174,000,000 USD, 274,515,000,000 USD)
          = 22,701,500,000 USD
Step 3: Compare the claimed value to the tolerance range.
Tolerance range: -22,701,500,000 USD to 22,701,500,000 USD
Since the claimed value falls within the tolerance range, the claim is consistent.
VERDICT: consistent
```

```
TRAINED MODEL (1,300 STEPS)

Step 1: Identify the relevant values from the reported data.
- Revenue for period ending 2020-09-26: 274,515,000,000 USD
- Revenue for period ending 2019-09-28: 260,174,000,000 USD
Step 2: Calculate the actual change in revenue.
Actual change = 274,515,000,000 USD - 260,174,000,000 USD
              = 14,341,000,000 USD
Step 3: Compare the claimed value with the calculated actual change.
Claimed value: 14,341,000,000 USD  |  Actual change: 14,341,000,000 USD
The claimed value matches the actual calculated change.
VERDICT: consistent
```

The base model's arithmetic typos a number mid-calculation (`2260,174,000,000` instead of `260,174,000,000`) yet still lands on the right difference — the final number reads recalled, not derived. It also checks the claim against a symmetric tolerance band around zero, a different and weaker test than what it's actually supposed to check, and only passes here because the claimed change happens to be small relative to that band. The trained model computes the actual change cleanly and compares it directly to the claim — the same check the real Reconciler performs. That's the shape of improvement behind the numbers above.

WHAT IT TOOK TO GET HERE

Three real upstream packaging bugs, none about the reward design itself: TRL's `GRPOTrainer` unconditionally imports an unmaintained, incompatible `llm_blender` and hits a `weave` availability check that false-positives in this environment, for judge/logging features never used here — worked around with a `sys.modules` stub rather than chasing dependency versions for functionality the project doesn't touch. It also assumes a `model.warnings_issued` attribute the PEFT/Unsloth-wrapped model doesn't initialize — fixed with a one-line defensive guard. Full writeup, including the first (inconclusive, honestly reported) 150-step run: `docs/phase7-results.md`.

RETRAINED ON THE FIXED DATASET — THE HONEST RESULT

Finding 09 below traces `bps_change`'s weak performance to a data-generator bug, not a reasoning limitation — the dataset had only 2 real distinct ratios to draw from. Fixed and retrained for real on the same box: 3,403 examples (up from 1,612), same architecture, same 1,300 GRPO steps, evaluated on a fresh 706-example held-out split (not comparable row-for-row to the 304-example table above — a different split of a larger dataset).

	BASE	TRAINED	DELTA	NOISE FLOOR
Overall accuracy	**0.820**	**0.858**	**+0.038**	**±0.038**
False-consistent rate	0.108	0.020	−0.088	±0.025
absolute	0.969	0.994	+0.025	±0.021
absolute_change	0.667	0.825	+0.158	±0.108
growth_pct	0.858	0.893	+0.035	±0.070
bps_change	0.436	0.372	−0.064	±0.140

Held-out test set, n=706. Noise floor: combined 95% half-width across the two proportions.

The false-consistent rate improved again, by a wide, noise-floor-clearing margin (0.108 → 0.020) — the dangerous failure mode kept shrinking on a second, independent run. `absolute_change` improved substantially and really (+0.158). But the honest headline is `bps_change` itself: the fix that tripled its raw training-example count (114 → 350) produced **no measurable win** — the number actually moved

down slightly (0.436 → 0.372), though well inside the noise floor at this small stratum (n=94), so not proven worse either, just not proven better. A plausible reason, not yet confirmed: `max_steps` stayed fixed at 1,300 while the dataset grew 2.1× — so despite the raw `bps_change` count tripling, every example (that category included) got proportionally *less* repeated exposure than in the original run. If that's right, the next real experiment is more steps on the bigger dataset, not the reproportioned data alone — flagged as follow-up work, not run here.

> **TESTED THE STEP-BUDGET HYPOTHESIS DIRECTLY — IT DIDN'T HOLD**
>
> Ran the follow-up flagged above: same dataset, same architecture, `max_steps` scaled from 1,300 to **2,730** to match the dataset's 2.1× growth, same 706-example held-out split. Result: overall accuracy `0.820 → 0.865` and false-consistent rate `0.108 → 0.024`, both real again; `absolute_change` reached `0.667 → 0.867` — the largest clean win across all three runs. But `bps_change` moved to only `0.404` — numerically closer to the 0.436 baseline than the 1,300-step run's 0.372, but that move is itself inside the noise floor against the 1,300-step result (±0.139), so not a real change either. Doubling the steps didn't fix it. The step-budget-dilution hypothesis is the one that turned out to be wrong here — whatever's actually holding `bps_change` back isn't simply a matter of more repeated exposure, reported as the real, if less satisfying, answer rather than left as an open theory that sounded plausible.

03 A real run, at scale

Not a cached demo — this is the actual coordinator, run against NVDA's FY2026 10-K, live EDGAR data and a live LLM call for every extraction.

NVDA · FY2026 10-K · MD&A		**13 claims verified**
CONSISTENT	Revenue EDGAR: $215,938,000,000 — 0.02% off	$215.9B
INCONSISTENT	Revenue growth EDGAR implies 46.09% growth, not the stated figure	65.0%
CONSISTENT	Income tax expense EDGAR: $21,383,000,000 — 0.08% off	$21.4B
UNVERIFIABLE	Data Center revenue growth segment-level — no top-level XBRL tag exists; declined rather than guessed	68.0%

13	22	1.39s	0
claims checked	tool calls	wall-clock	crashes

One filing is a worked example, not a scale claim. Phase 6 runs the same real coordinator — no mocks — against 5 companies' real 10-Ks, including GOOGL and AMZN, deliberately never used while building any of this, to check whether the pipeline generalizes or was quietly tuned to the three it had already seen.

5/5	18	128	0
tickers succeeded	claims verified	tool calls	crashes

Budget-capped per ticker (25 chunks / 20 extraction calls / 240s) — a real 10-K can run 100+ MD&A chunks, each an LLM call.

The first run surfaced two real coverage gaps rather than a clean pass — the kind of thing three hand-picked companies can hide. GOOGL and AMZN returned *zero* claims: their real body heading for "Item 7. Management's Discussion..." renders as an isolated `"Item 7."` node with the title following separately, structurally identical

to what the parser had been treating as a table-of-contents-only pattern — a heuristic that looked solid against 3 companies and broke on the 4th. Fixed by scoring every candidate heading window by the gap to its nearest end-marker instead of by which text node it landed in (a real section runs hundreds of lines; a ToC entry is a few). Separately, most real MD&A prose past the top-line numbers is segment-specific ("Azure revenue," "H20 charge," "XBOX content and services revenue") — investigated rather than assumed fixable: SEC's company-facts API returns every data point as a flat record with no segment/member/axis field at all, so those claims correctly decline rather than guess. But not everything in that unresolved list was actually segment-level — "Commercial remaining performance obligation" turned out to map to `RevenueRemainingPerformanceObligation`, a real top-level concept just missing from the dictionary. Re-run after both fixes: MSFT's remaining-performance-obligation claims now resolve to real verdicts instead of "unverifiable" — `CONSISTENT` on the absolute figure (0.88% off), `INCONSISTENT` on the growth figure, itself flagged as possibly a known period-matching limitation rather than a real miss.

> **THE OTHER HARNESS CLAIM, EXERCISED LIVE TOO**
>
> Budget enforcement shows up in every run above (every ticker hit its cap). Checkpoint/resume — the other half of the harness's real-conditions claim — had only ever been proven against mock scenarios. A dedicated run against MSFT's real 215-chunk filing closes that: a tiny budget forces a stop at chunk 3, a real checkpoint lands on disk, a resume loads it, skips re-extracting those 3 chunks, and continues forward to chunk 23 with 11 real claims accumulated — the save → load → skip → continue cycle, proven against a live filing, not a fixture.

04

What actually broke

The interesting engineering here wasn't the happy path — it was what real SEC data did to the happy path. Each of these was caught by testing against live data, not by trusting the design on paper.

01

A metric-matching shortcut that would verify the wrong number

Resolving a claim's free-text metric ("Azure revenue") to an exact XBRL concept originally tolerated wording variance via substring matching. But `"revenue"` is a substring of `"Azure and other cloud services revenue"` — so a segment's revenue would have been silently checked against *total company* revenue and returned a confidently wrong "consistent" verdict. Fixed to exact-match plus explicit aliases only.

`agents/resolver.py`

02

XBRL doesn't tag percentages as percentages

Rate concepts like effective tax rate are reported with `unit="pure"` — a decimal fraction (`0.19` , not `19`). A claim stating "19%" would have failed to match the fact at all, and once that was fixed, would have compared `19.0` against `0.19` and read as wildly wrong even when correct.

`agents/verification_agent.py`

03

Companies retag their own concepts mid-history

NVDA reported revenue under `RevenueFromContractWithCustomerExcludingAssessedTax` through FY2022, then switched to plain `Revenues` . The old tag never disappears from the company's concept list — it just stops receiving new data. Picking "the first candidate that exists at all" silently locked onto a tag with no data newer than 2022. Fixed to pick by data recency instead.

`agents/resolver.py`

04

A newer filing can corrupt an older one's claims

A claim extracted from a 10-K's MD&A describes *that 10-K's* fiscal year — but a newer 10-Q, almost always already filed by the time verification runs, has a more recent quarterly `period_end`. Without a cutoff, that quarterly figure would get picked as "current," comparing the wrong two numbers entirely. Fixed with an `as_of` anchor tied to the claim's own source filing.

`agents/resolver.py`

05

A bug in the measurement itself, caught before it could lie

An early precision/recall scorer would have counted a generic prediction of `"Revenue"` as matching gold label `"Microsoft Cloud revenue"` — a bug in the *grader*, which is worse than a bug in the thing being graded, because it makes bad results look good. Its own unit test caught it before the real comparison ever ran.

`eval/metrics.py`

06

Reasoning silently corrupted its own numeric output

Switching the extraction signature from `dspy.Predict` to `dspy.ChainOfThought` made the model write dollar amounts in shorthand inside its reasoning — *"$215.9 billion"* — then just echo that literal `215.9` into the structured `value` field instead of expanding it to `215900000000`. Consistent, every run. Plain `Predict` never had this failure mode, because it never generates that intervening shorthand text to anchor on. Fixed by instructing the signature to write the fully-expanded number in the reasoning itself, so the structured step has nothing left to get wrong.

`eval/dspy_extractor.py`

07

A restatement cutoff that resolved periods but not values

The verification agent already anchored *period and concept* resolution to an `as_of` cutoff — the claim's own source filing date — so a 10-Q couldn't leak into a 10-K's period selection (finding 04). But that same cutoff never reached the reconciler's own fact lookup: a claim from an older filing could still be compared against a company's *later restatement* of that same period and get flagged inconsistent, even though it was accurate when made. Fixed by threading `as_of` through `reconcile()` itself, with a regression test built from a live-numbers reproduction of the bug.

`tools/reconciler.py`

08

Auditing the reward function before it has a job

The Reconciler is the ground-truth signal for Phase 7's RLVR/GRPO reward — which means an exploitable seam in it wouldn't just add noise, it would give policy optimization something specific to find and amplify. Before any GPU time is spent, `eval/reconciler_audit.py` now runs 15 known-good, known-bad, and adversarial cases (sign flips, magnitude confusion, comparison-type mislabeling, tolerance-boundary probes) as a permanent CI check. Result: 15/15 correct verdicts, zero false-"consistent" outcomes — the one failure mode worth watching for, since a false alarm is costly but a missed inconsistency is the dangerous direction.

`eval/reconciler_audit.py`

09

A weak category traced to the data generator, not the model

`bps_change` stayed the worst RLVR category through training (0.233 → 0.400) — the easy read is "the hardest reasoning task, needs more steps." The actual cause was upstream: the dataset generator only had *2 real distinct ratios* to draw from (gross margin, operating margin — 2 of its original 4 "pairs" were the same ratio under a company's alternate revenue tag, not a second ratio), which made `bps_change` 8.7% of training data against 43.3% for `absolute`. Not rare reasoning — rare *examples*. Fixed by adding 5 more ratio pairs (net margin, R&D intensity, SG&A ratio, cost ratio, opex ratio) from concepts already verified present in real company data: `bps_change`'s share rose to 13.0%, raw count 114 → 350. Retrained on it for real (§02) — the honest result is a null, not a win: `bps_change` didn't clear its own noise floor either direction, most plausibly because the GRPO step budget didn't scale with the larger dataset. A cleanly traced root cause, fixed, and still not sufficient on its own — the kind of finding worth keeping, not smoothing over.

`phase7/build_dataset.py`

10

A claim's own stated period was being ignored entirely

`resolve_periods()` always picked the globally most-recent fact as "current," regardless of what the claim's own period text actually said — so two claims from the same sentence naming two different fiscal years ("Income tax expense was $21.4 billion and $11.1 billion for fiscal years 2026 and 2025, respectively") both resolved to the *same* (current, comparison) pair, and the FY2025 claim ended up checked against the FY2026 fact. Confirmed against this exact live NVDA sentence: without a hint, the FY2025 claim's "current" resolved to the FY2026 annual figure ($21.38B) against a $13.9B quarterly comparison — nonsensical. Fixed by threading the claim's own period text through as a hint: an explicit fiscal year now picks that year's fact as current, and "sequentially" vs. "a year ago" now correctly distinguish the immediately-preceding period from the same period ~12 months back. Re-run against the same live sentence: both claims now independently resolve `CONSISTENT` against their own correct periods.

`agents/resolver.py`

11

A quarter reference resolved to the wrong duration entirely

A named ordinal quarter ("the third quarter", "Q3") had been deliberately left unresolved — real filings also name quarters by calendar month ("the September quarter"), and guessing that mapping wrong without knowing a company's fiscal-year-end risks a wrong-but-confident match. The ordinal form is safe, though: XBRL's own `fiscal_period` field is itself fiscal-quarter-numbered, so "the third quarter" maps directly to `fp="Q3"` with no guessing involved. Implementing it surfaced a second, sharper bug before it shipped: a 10-Q commonly tags *both* the standalone 3-month figure and a 6-/9-month year-to-date cumulative with the identical `fiscal_period` and `period_end` — an unfiltered match against real NVDA data returned a `$91.166B` "Q3" figure that was actually the 9-month cumulative, not the `$35.082B` standalone quarter. Fixed by preferring the ~91-day-length fact when both exist. Calendar-named quarters are still correctly declined rather than guessed.

`agents/resolver.py`

OPEN, NOT HIDDEN

A quarter named by calendar month ("the September quarter", "the December quarter") still isn't parsed into a specific fiscal period — translating that safely needs the company's own fiscal-year-end, which isn't available at this resolution step, so it still falls back to the default "most recent" pick rather than guessing. Flagged in `agents/resolver.py` as follow-up, not silently left broken.

05 A real backtest: would this have caught an actual restatement?

Every result above is checked against real filings, but always in the present tense — does the pipeline verify a claim correctly *today*? A sharper question: has this project's own bitemporal machinery ever actually caught something real happening to a real company? SEC 8-K "Item 4.02" disclosures — a company telling investors its own past financial statements can no longer be relied on — supply real ground truth to test that against, using nothing synthetic.

13,827	541	122	118/122
real restatement fingerprints found	companies, from real 8-K filings	matched to real MD&A prose, 16 companies	flagged inconsistent — 96.7%

Three stages, each verified before moving to the next. **Finding restatements:** SEC's own XBRL company-facts data keeps both the original and the corrected value for the same (concept, period) when a company amends a filing — group by (concept, period), keep only groups where a `10-K/A` or `10-Q/A` specifically reports a different value. A looser first pass (any later-filed value counts) found 42,200 "fingerprints" that fell apart on inspection — 82% were routine reclassifications, not corrections. Requiring the later value come from an actual amendment dropped that to a defensible 13,827, across 541 companies. **Matching to prose:** for the top 60 of those companies by fingerprint magnitude, fetch the *original* filing's MD&A and run the project's real extraction agent over it, keeping claims whose metric resolves to the same concept and whose value is within 5% of the fingerprint — prose commonly rounds ("$1.8 billion" for $1,838,000,000). 409 filings tried, 122 real claims matched this way, across 16 companies — Discover Financial Services and Rithm Capital Corp. (the pair the reliability fix below was first proven against) plus AZEK, Seneca Foods, Camber Energy, Astrana Health, Sanmina, Compass Minerals, and nine more. **The backtest itself:** for each matched claim, run the actual `reconcile()` function — the same code every other result in this document uses — twice: once with the bitemporal `as_of` cutoff set to the claim's own filing date (what did the reconciler know when the claim was made?), once set to the restated filing's date (what does it know now?). No LLM calls in this step — pure arithmetic against already-known claim values.

A sample of the 122 matched claims, illustrating all outcomes	as filed → after restatement
→ FAIL	Discover Financial Services — net income, Q3 2023 consistent at filing (0.0% off) → inconsistent once restated (16.55% off)	$683M claimed
→ FAIL	Camber Energy — net income, Q2 2023 consistent at filing (0.0% off) → inconsistent once restated (250.53% off — one of the more dramatic corrections in the set)	$1.92M claimed
FAIL → FAIL	Discover Financial Services — net income, Q1 2022 already inconsistent at filing (3.38% off) — the prose's own rounding missed tolerance before any restatement existed	$1.2B claimed
PASS → PASS	Discover Financial Services — net income, FY2021 stayed consistent even after restatement — the 1.12% correction was smaller than the prose's own rounding, both sides land inside the 1% tolerance	$5.4B claimed

102 of 122 show the clean target pattern — consistent when the claim was made, inconsistent once the restatement existed, using only data that existed at each point in time. 16 more were already inconsistent at the original filing, for an unrelated reason: the prose's own rounding fell outside tolerance regardless of any later restatement — still a real catch, just not evidence the bitemporal mechanism specifically did the work. 3 are structurally unverifiable on both sides — `absolute_change` claims ("decreased $33.4 million") where Phase B's matching step doesn't capture the comparison period this claim type needs, so `reconcile()` correctly declines rather than guessing one. 1 of 122 is a genuine miss, reported rather than hidden: a real 1.12% restatement that happened to sit inside the gap between the prose's rounding and both the original and corrected figures. Combined, 118 of the 119 checkable matches (99.2%) — 118 of all 122 (96.7%) — would have been flagged inconsistent by the time each restatement existed — not a synthetic eval number, a real outcome across 16 real companies' real regulatory history.

SCOPED, NOT EXHAUSTIVE

122 matches come from 16 companies out of the 541 with real restatement fingerprints — the top 60 by fingerprint magnitude, not the full population. A first full-scan attempt across those 60 stalled past two hours with no incremental saving, a real reliability bug fixed with hard per-chunk timeouts and a save after every filing; the fix was proven on 2 companies first, then run for real across all 60 (moved to an always-on box partway through, since the run outlives a laptop staying open). 13,827 fingerprints and 541 companies are counted directly from real SEC data; 122 is the number actually pushed all the way through prose-matching and backtesting, stated as what it is rather than rounded up to the larger pool it was drawn from.

06

Why build a deterministic reconciler instead of just asking an LLM?

Every result above proves this *pipeline* works. This tests whether the *architecture choice* did — structured extraction, exact concept resolution, then deterministic bitemporal arithmetic, instead of the obvious alternative: hand a general LLM the claim and the raw numbers and let it decide. Tested directly against two already-verified test sets, same LLM endpoint used everywhere else in this project, no new hand-labeling.

	DETERMINISTIC RECONCILER	NAIVE LLM BASELINE
Adversarial audit (15 cases)	**15/15 (100%)**	**11/15 (73.3%)**
Backtest, correct flip pattern	102/122 (83.6%)	50/122 (41.0%)
False-positive rate at original filing	0% (by construction)	49% (49/100)

On `eval/reconciler_audit.py`'s 15 adversarial cases (each already 100% correct for the deterministic reconciler, by construction), the naive baseline is handed the exact same facts and asked to judge via reasoning alone: **11/15 (73.3%)**. The miss worth sitting with: a claim off by **1,000,000×** (stated in millions against facts reported in raw dollars) read as `CONSISTENT` to the naive baseline — the exact dangerous failure mode this whole project is built around, and one plain division in `tools/reconciler.py` never gets wrong.

The sharper test: the 122-match real backtest, checking whether a naive read can do the bitemporal reasoning the `as_of` design exists for. Given the *raw, unfiltered* fact history — every filed value, not the one bitemporally-correct value `_find_fact()` would pick — and told the claim's filing date in plain English, the naive baseline reproduced the correct "consistent when made, inconsistent once restated" pattern in only **50 of the 102 cases (49%)** the deterministic pipeline got right. Worse, and more precise: of the 100 real cases that were genuinely consistent as of their own original filing, the naive baseline **falsely called 49 of them "inconsistent"** — bleeding the restatement's later information backward into a judgment about the past. Telling a model "only use data filed on or before this date" in a prompt does not reliably make it do that; writing the cutoff into the fact-lookup code does — the same lesson this project's own development history already learned once, in finding 07.

HONEST SCOPE

Run against this project's own self-hosted `gpt-oss-20b` endpoint, held constant to isolate "does structure help" from "is this specific model good at this" — a stronger commercial model would plausibly narrow the gap, especially on raw arithmetic, and is a real follow-up not run here. One prompt attempt per case, no retries or self-consistency voting. A real harness bug was caught and fixed while building this too: the first pass omitted the denominator concept's facts for `bps_change` cases, so the naive baseline correctly said "unverifiable" on data it was never shown — fixed before the numbers above were reported.

07 Does it cry wolf on clean filings?

The backtest only tests sensitivity — it shows the reconciler flags claims that turned out to be wrong, on companies that actually restated. Never checked the other way: run against companies that *didn't* restate, does it stay quiet on genuinely accurate claims? The real Coordinator, no mocks, against 8 large tech companies confirmed to carry no restatement fingerprint anywhere in Phase A's scan.

137	18	6	0/24
real claims, 8 companies	consistent	inconsistent	reconciler false positives, given correct inputs

An apparent 25% false-positive rate among the 24 resolved verdicts — worth investigating before reporting, not reporting at face value. It split into two real, distinct causes. The first pass actually found **14** inconsistent claims (58.3%). Digging into the largest cluster found a genuine, previously undiscovered bug: `resolve_periods()`'s *default* comparison-period pick (no explicit hint) had no check that the comparison period's duration matched the current one's. Confirmed against real TXN data — the company's own MD&A states plainly *"Revenue of $17.68 billion increased $2.04 billion, or 13.0%, ... compared to fiscal 2024"*, an accurate claim — but the resolver picked a Q3 2025 10-Q's **9-month year-to-date** fact ($13.259B) over the correct FY2024 annual figure ($15.641B), because the YTD fact's `period_end` sorted more recently. **Fixed**, with regression tests from the real numbers: 8 of the 14 original false positives resolved correctly afterward.

The remaining 6 trace to extraction, not the reconciler. Three TXN claims show the identical, unambiguous pattern:

QUOTE	CLAIMED	REAL (CURRENT PERIOD)
"Net income was $5.00 billion compared with $4.80 billion."	$4.80B	$5.001B
"...provision for income taxes was $709 million compared with $654 million."	$654M	$709M
"...effective tax rate...was 12.4% in 2025 compared with 12.0% in 2024."	12.0%	12.4%

Every one is a sentence shaped "[current value] ... compared with [prior value]" — extraction consistently grabbed the *second* number, not the first. A real, specific failure mode, distinct from anything in §04, and squarely `eval/dspy_extractor.py` territory — not fixed here. CSCO's and CRM's remaining misses look like the same class of extraction inaccuracy but weren't traced as cleanly, and that's stated honestly rather than overclaimed.

> **THE NUMBER THAT ACTUALLY MATTERS**
>
> Given the maker/checker separation this project is built around, the reconciler's own precision — given *correct* inputs — is the number worth trusting, not a blended rate that conflates extraction and verification errors. **Reconciler false-positive rate given correct inputs: 0/24.** Every resolved verdict is either a real match, or a real mismatch the reconciler correctly caught — 8 because of the period-selection bug found and fixed here, 6 because extraction handed it a wrong number. In no case did the reconciler itself produce an incorrect verdict against an accurate claim and correctly-resolved data. The 25% system-level rate is real, but it's an extraction-accuracy number wearing a specificity label.

A third bug turned up narrowing this control set to one vertical rather than expanding it. INTU returned zero claims — not assumed to be the same coverage gap as the financials/consumer-staples batch, checked directly. Its real heading reads *"ITEM 7 - MANAGEMENT'S DISCUSSION AND ANALYSIS..."* — dash-separated, no period — and the parser's regex only tolerated an optional period. **Fixed**, with regression tests from the real heading text: INTU now returns 280 real MD&A chunks instead of 0. IBM's zero claims turned out not to be a bug at all — its 10-K states plainly *"Refer to pages 6 through 38 of IBM's 2025 Annual Report to Stockholders, which are incorporated herein by reference"* — the MD&A simply isn't in this document, a real and legitimate filing pattern, not something a heading-regex fix could reach.

08 What would full coverage actually cost?

The README's "why now" argument is directional — falling inference cost makes full coverage a cost problem, not a capability one. Turned into a real number, not a claim. Verification is free (pure Python arithmetic against already-fetched XBRL facts); retrieval is free (plain HTTP GETs); extraction is the only step with a real dollar cost, so it's the only one measured.

1,537	454	215	$11.31
avg prompt tokens / call, measured	avg completion tokens / call, measured	chunks, MSFT's real 10-K MD&A	all 500 S&P 500 companies, one year

Real, cache-disabled extraction calls (a unique nonce per chunk rules out a stale cached response standing in for a genuine one) against MSFT's real MD&A, at `gpt-oss-20b`'s published API pricing — the exact model this project's own endpoint runs — $0.030/1M input, $0.130/1M output tokens, checked directly rather than assumed from memory. Eleven dollars a year for extraction across every S&P 500 company's full annual disclosure — the whole index, not one company. The prompt cost is dominated by fixed instruction overhead (~1,500 tokens), not chunk size, so cost scales with chunk *count*, not filing length, in this range.

> **WHAT THIS NUMBER DOESN'T SAY**
>
> It's inference cost at the current, un-fine-tuned model — not a claim about accuracy or coverage. Phase 7's own results (§02) show real reasoning gaps a zero-shot model has; §07's specificity check and `docs/robustness-and-scope.md` show real coverage gaps in what fraction of real claims resolve today. $11.31 buys running the pipeline over every chunk, not verifying every claim a filing makes — the honest claim this number supports is narrower than "full coverage is solved."

Written up as part of an ongoing capstone build.

View the repository →